

บทที่ 2

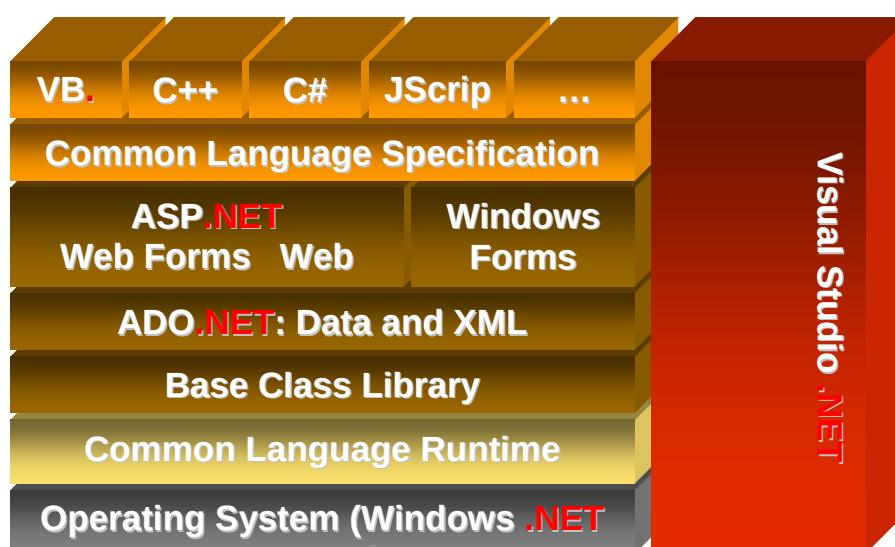
Microsoft.NET

2.1 รู้จักกับ Microsoft.NET

Microsoft.NET หรือเรียกสั้นๆว่า .NET เป็นเทคโนโลยีใหม่ล่าสุดที่พัฒนาขึ้นโดยบริษัทไมโครซอฟท์ .NET คือแพลตฟอร์มในการพัฒนาซอฟต์แวร์สำหรับระบบปฏิบัติการ Windows โดยถือเป็นก้าวสำคัญในการพัฒนาซอฟต์แวร์ยุคใหม่ซึ่งนำเสนอหลักการพัฒนาซอฟต์แวร์ด้วยภาษาอะไรก็ได้ที่เราถนัด และสามารถเรียกใช้งานโปรแกรมที่เขียนด้วยภาษาอื่นๆได้อย่างกลมกลืน ใน .NET มีภาษาโปรแกรมใหม่ๆเกิดขึ้น เช่น Visual Basic.NET, C#.NET, C++.NET, J#.NET หรือแม้กระทั่ง COBOL.NET เป็นต้น

2.2 สถาปัตยกรรม.NET Framework

สิ่งที่เป็นหัวใจสำคัญของ .NET ก็คือ .NET Framework ซึ่งมีส่วนประกอบต่างๆดังแสดงในรูป



รูปที่ 2-1 รูปแสดงสถาปัตยกรรม .Net Framework

.NET ไม่ได้เป็นเพียงแค่ภาษาโปรแกรมใหม่เท่านั้น แต่ยังถือเป็นรากฐานในการพัฒนาแอปพลิเคชันยุคใหม่ ความสามารถที่โดดเด่นของ .NET ก็คือการที่เราสามารถพัฒนาโปรแกรมด้วยภาษาใดๆก็ได้ที่สนับสนุน Common Language Specifications (CLS) ของ .NET ซึ่งจะช่วยให้ นักพัฒนาสามารถเลือกใช้ภาษาโปรแกรมใดๆก็ได้ (language neutral) โดย .NET Framework จะมีเครื่องมือที่เรียกว่า Visual Studio .NET ซึ่งถือเป็น Integrated Development Environment (IDE) สำหรับการพัฒนาโปรแกรม

โปรแกรมที่เราเขียนขึ้นมานั้นเมื่อทำการคอมไพล์ (Compile) แล้วจะอยู่ในรูปของ intermediate language ที่เราเรียกว่า MSIL (Microsoft Intermediate Language) ซึ่งเป็นแนวคิดเดียวกันกับ “ไบต์โค้ด” ของ Java Platform นอกจากนั้นโปรแกรมที่พัฒนาขึ้นภายใต้ .NET จะสามารถเรียกใช้งานโปรแกรมที่เขียนด้วยภาษาอื่นได้ถ้าหากภาษานั้นอยู่ภายใต้มาตรฐาน CLS เหมือนกัน ปัจจุบันมีภาษาโปรแกรมมากกว่า 20 ภาษาที่สนับสนุน CLS เช่น Pascal.NET, Perl.NET หรือแม้กระทั่ง COBOL.NET เป็นต้น

ใน .NET เราสามารถสร้างแอปพลิเคชันได้หลากหลายรูปแบบ ได้แก่ Win Form, Web Form และ Web Service สำหรับ Win Form หรือ Windows Form นั้นคือการพัฒนาโปรแกรมบน Windows ทั่วไป, Web Form คือการพัฒนาเว็บแอปพลิเคชัน ซึ่งการพัฒนาเว็บแอปพลิเคชันด้วย .NET จำทำได้ด้วยวิธี drag-and-drop เช่นเดียวกับการพัฒนาโปรแกรมบน Windows และสุดท้ายคือ Web Service เป็นการพัฒนาแอปพลิเคชันแบบใหม่ซึ่งมองแอปพลิเคชันเป็น encapsulate ไว้ในรูปของคลาสเช่นเดียวกัน

โปรแกรมที่พัฒนาด้วย .NET จะมีการเรียกใช้ข้อมูลประเภทเดียวกันทั้งหมดไม่ว่าจะเขียนด้วยภาษา C#.NET, VB.NET หรือภาษาอื่นใดก็ตาม ประเภทข้อมูลเหล่านี้จะอยู่ในกลุ่มของคลาส (Class) Data และ XML เพื่อใช้ในการเรียกใช้และจัดการฐานข้อมูลหรือข้อมูลในรูป XML เป็นต้น

ส่วนชั้น Base Class เป็นที่รวมของคลาสพื้นฐานต่างๆ ซึ่งไมโครซอฟท์พัฒนาขึ้นมาให้สามารถเรียกใช้งานและพัฒนาต่อยอดเพิ่มเติมได้ ซึ่ง Base Class นี้ครอบคลุมถึงสิ่งที่จำเป็นในการพัฒนาโปรแกรม เช่น การจัดการอินพุต/เอาต์พุต การจัดการฐานข้อมูลในรูป XML เช่นคลาส ADO.NET, XML เป็นต้น

ชั้นสุดท้ายซึ่งเป็นส่วนสำคัญของ .NET Framework ได้แก่ Common Language Runtime (CLR) ถือเป็นรากฐานของแพลตฟอร์ม .NET เลย์ที่เดียว หน้าที่ของ CLR ก็คือเป็น execution engine ในการประมวลผลและจัดการ โปรแกรมที่คอมไพล์แล้วให้ทำงานได้บนระบบปฏิบัติการ Windows โดย CLR จะแปลงโค้ดในรูป MSIL ไปเป็นคำสั่งภาษาเครื่อง (machine language) โดยใช้เทคโนโลยีการแปลงแบบ Just-In-Time (JIT) คือแปลงเฉพาะส่วนที่จะนำมาใช้เท่านั้น หลังจากนั้นถ้าต้องการนำส่วนอื่นมาใช้งานอีกก็จะแปลงเพิ่มเฉพาะในส่วนนั้น ซึ่งช่วยให้โปรแกรมทำงานได้เร็วขึ้นเนื่องจากไม่ต้องรอให้แปลงเสร็จทั้งหมดก่อนจึงจะทำงานได้ นอกจากนี้ CLR ยังทำหน้าที่ติดต่อกับระบบปฏิบัติการ, จัดสรรหน่วยความจำให้กับโปรแกรมต่างๆ และคืนหน่วยความจำที่ไม่ถูกใช้งานแล้วให้กับระบบ (ด้วยกระบวนการที่เรียกว่า Garbage Collection), จัดการกับข้อผิดพลาด (exception handling) รวมถึงดูแลเรื่องความปลอดภัย (security management) ด้วย

2.3 ADO.NET

ADO.NET มาจากคำว่า ActiveX Data Object .NET ทำหน้าที่รับผิดชอบในส่วนของการจัดการข้อมูลในฐานข้อมูลประเภทต่างๆ ของสถาปัตยกรรม .NET โดยอาศัยเทคโนโลยี OLEDB Data Provider เป็นตัวกลางในการเชื่อมต่อกับฐานข้อมูลแต่ละประเภท

สถาปัตยกรรมของ ADO.NET ถูกออกแบบมาเป็นสถาปัตยกรรมแบบ disconnected ซึ่งหมายถึงว่า แอปพลิเคชันจะถูกเชื่อมต่อกับฐานข้อมูลเฉพาะให้นานเพียงพอที่จะรัน และอัปเดตข้อมูล และก็ตัดการเชื่อมต่อออกไป เมื่อรันเสร็จ

ข้อมูลออกมาแล้ว คุณสามารถกำหนดจัดเก็บมันไว้บนออบเจกต์ข้อมูลของ ADO.NET ได้ เช่น DataSet หรือ DataView

การใช้สถาปัตยกรรมแบบ disconnected มีข้อดีอยู่มากมาย ข้อดีที่สำคัญที่สุดอย่างหนึ่ง ก็คือ ทำให้แอปพลิเคชันมีขนาดการรองรับได้มากขึ้น ซึ่งหมายถึงว่า ฐานข้อมูลของคุณจะทำงานรองรับผู้ใช้เป็นร้อย ๆ รายได้เหมือนกับว่ารองรับผู้ใช้เพียงคราวละสิบรายเท่านั้น เนื่องจากฐานข้อมูลมันถูกเชื่อมต่อไว้นานเพียงพอที่จะให้มีการค้นคืน และอัปเดตข้อมูลในแต่ละครั้งเท่านั้น แล้วก็จะปล่อยคืนพร้อมให้อินสแตนซ์อื่นของแอปพลิเคชันของคุณเข้าทำงานต่อไป

2.3.1 โครงสร้างการทำงานใน ADO.NET

ผู้ที่เคยใช้ Visual Basic 6.0 พัฒนาโปรแกรมด้านฐานข้อมูลมาก่อนจะทราบว่าเราใช้เทคโนโลยี ADO (ActiveX Data Object) เป็นเครื่องมือติดต่อและจัดการข้อมูลในแบบ connected database ก็จะต้องเปิดการเชื่อมต่อกับฐานข้อมูลไว้ตลอดช่วงเวลาที่ใช้งานซึ่งจะทำให้การเพิ่ม ลบ แก้ไขหรือการกระทำใดๆก็ตามที่เกี่ยวข้องกับข้อมูลจะเป็นการกระทำกับข้อมูลจริงในไฟล์ฐานข้อมูลนั้นโดยตรง ซึ่งมีข้อดีคือทำให้ข้อมูลถูกปรับปรุงให้ทันสมัยและถูกต้องอยู่ตลอดเวลา แต่ข้อเสียคือ ถ้ามีผู้ใช้งานข้อมูลเป็นจำนวนมากและพร้อมๆกันหรือการประมวลผลข้อมูลที่มีปริมาณมากจนทำให้ระยะเวลาโดยรวมของระบบลดลงได้ ดังนั้น Visual Studio .Net จึงได้นำเทคโนโลยีตัวใหม่ล่าสุดคือ ADO.NET ที่พัฒนาจาก ADO มาใช้จัดการข้อมูลแบบ disconnected database คือสามารถจัดการข้อมูลในฐานข้อมูลโดยไม่ต้องเปิดการเชื่อมต่อไว้ตลอดเวลาเหมือน ADO เนื่องจากข้อมูลจะถูกอ่านจากฐานข้อมูลมาเก็บไว้ในหน่วยความจำก่อน จากนั้นจะตัดการเชื่อมต่อกับฐานข้อมูล ทำให้การกระทำใดๆหลังจากนี้จะเป็นการกระทำข้อมูลในหน่วยความจำเท่านั้น เมื่อเสร็จสิ้นกระบวนการแล้วจึงจะบันทึกกลับลงในฐานข้อมูล ซึ่งจะช่วยประหยัดทรัพยากรของระบบและเพิ่มประสิทธิภาพการทำงานของ SERVER (ที่จริงใน ADO เองก็มีความสามารถในการทำ disconnected database) ได้ แต่โปรแกรมเมอร์จะต้องเป็นผู้ควบคุมด้วยตนเองว่าจะให้มีการเชื่อมต่อหรือยกเลิกการเชื่อมต่อเมื่อใด ซึ่งต่างกับใน ADO.NET ที่จะตัดการเชื่อมต่อให้โดยอัตโนมัติ เมื่อไม่มีความจำเป็นต้องใช้ฐานข้อมูลในแบบ on-line และจะเปิดการเชื่อมต่อให้ใหม่เมื่อจำเป็นต้องทำงานร่วมกับฐานข้อมูลจริง)

2.3.2 DataSets และ XML

เมื่อคุณค้นคืนข้อมูลจากฐานข้อมูลที่ถูกจัดเก็บไว้ใน DataSet ซึ่งจะกล่าวถึงรายละเอียดในไม่กี่ปีก่อนนี้ โดยพื้นฐาน DataSet เป็นแคช (cache) ของข้อมูลที่ถูกจัดเก็บในหน่วยความจำ และอยู่ในรูป XML (Extensible Markup Language) ทำให้มีข้อดีอีกหลายประการ เช่น ทำให้คุณแชร์ข้อมูลกับแอปพลิเคชันอื่นได้ และทำให้คุณทำงานกับข้อมูลในหน่วยความจำได้ ซึ่งแน่นอนมันก็จะเข้าถึงได้รวดเร็วกว่าการค้นคืนโดยตรงจากฐานข้อมูล เนื่องจากข้อมูลอยู่ในรูป XML ไม่เพียงแต่คุณสามารถแชร์มันกับแอปพลิเคชันอื่น ๆ ได้เท่านั้น แต่มันยังเป็นแบบ self-describing ด้วย ทุกแอปพลิเคชันที่ซัพพอร์ต XML แม้ว่า คุณจะเขียนจากภาษาโปรแกรมต่าง ๆ ที่รันบนคนละเครื่องก็ยังสามารถใช้ข้อมูลที่คุณส่งให้ได้

2.3.3 System.Data Namespace และ OLE DB

ADO.NET ถูกออกแบบมาในภาพแวดล้อมพื้นฐานจาก System.Data namespace โดยสร้างจากเนมสเปซอื่น ซึ่งจะได้กล่าวถึงรายละเอียดต่อไป เนมสเปซเหล่านี้จะมีความสามารถที่ทำให้ ADO.NET เข้าถึงข้อมูลที่จัดเก็บไม่เพียงแต่ในฐานข้อมูลเท่านั้น แต่ยังสามารถเป็นข้อมูลใดที่ซัพพอร์ต OLE DB ด้วย OLE DB เป็นชุดของ COM interfaces ที่มีรูปแบบการเข้าถึงอย่างเดียวกันไม่ว่าจะเป็น data sources แบบใด ๆ data store ในแหล่งข้อมูลใด ๆ ที่จัดเก็บในแบบไม่ได้เป็นแบบเชิงสัมพันธ์ เช่น ไฟล์แบบ sequential

ทั้งหมดนี้หมายถึงว่าไม่เพียงแต่คุณสามารถเข้าถึงฐานข้อมูล เช่น Microsoft Access, Microsoft SQL Server หรือ Oracle แต่คุณยังสามารถเข้าถึงข้อมูลที่จัดเก็บในรูปแบบเอกสาร XML หรือแม้แต่ไฟล์เท็กซ์ที่จัดเก็บบนฮาร์ดไดรฟ์

2.3.4 ออบเจกต์ด้านฐานข้อมูลของ VB.NET

ออบเจกต์หลักของเทคโนโลยี ADO.NET ประกอบไปด้วยออบเจกต์ 3 กลุ่มใหญ่ๆ คือ

- กลุ่มออบเจกต์ที่อาศัย OLEDB Provider : ทำหน้าที่เข้าถึงข้อมูลในฐานข้อมูล มี 3 ตัวคือ ออบเจกต์

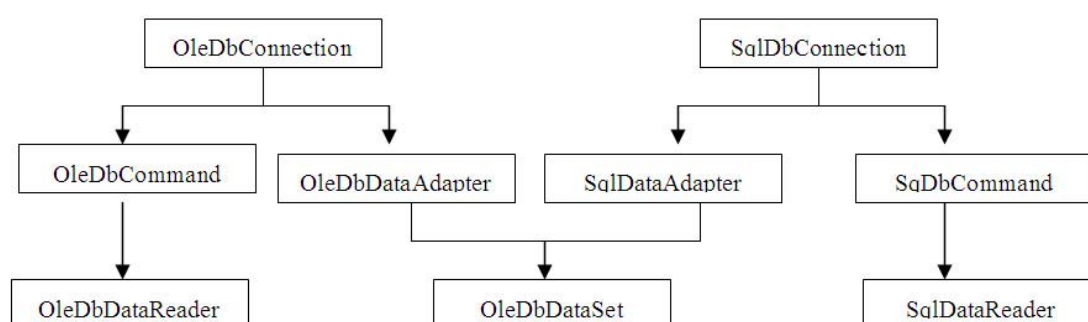
OleDbConnection, OleDbDataAdapter และออบเจกต์ OleDbCommand

- กลุ่มออบเจกต์ที่อาศัย MS SQL Server Provider : ทำหน้าที่เข้าถึงข้อมูลในฐานข้อมูล มี 3 ตัว คือ ออบเจกต์

SqlConnection, SqlDataAdapter และออบเจกต์ SqlCommand

- กลุ่มออบเจกต์ที่ใช้เก็บผลการทำงาน: มีอยู่ 3 ตัว คือ ออบเจกต์ DataSet, OleDbDataReader และออบเจกต์

SqlDataReader



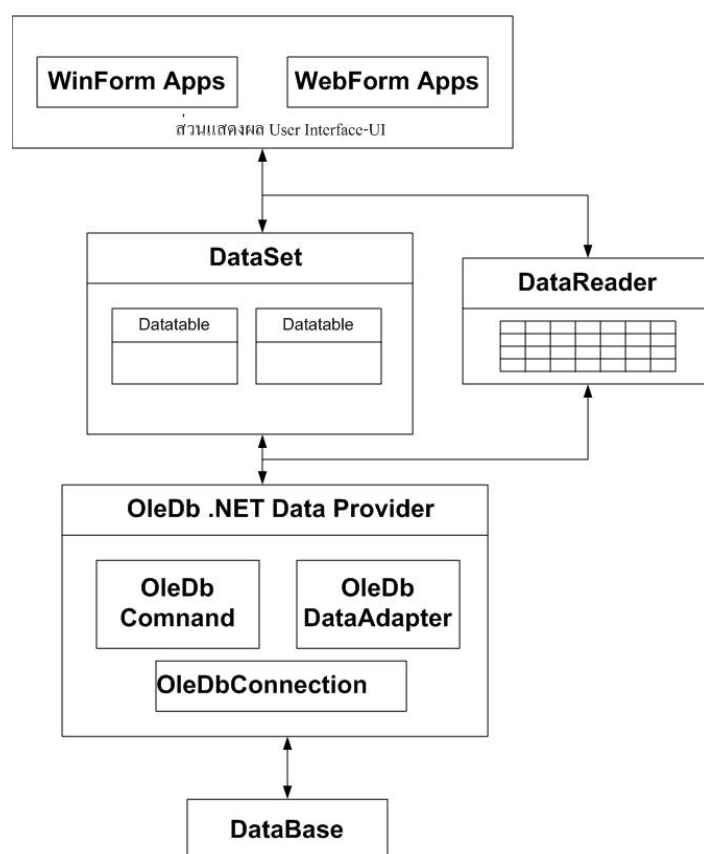
รูปที่ 2-2 รูปแสดงความสัมพันธ์ระหว่างออบเจกต์ของ VB.NET

จากรูป ไมโครซอฟท์ได้แยกกลุ่มออบเจกต์ ADO.NET ตามเทคโนโลยีการเข้าถึงข้อมูลออกเป็นมาตรฐาน 2 ส่วนอย่างชัดเจน

ในกรณีที่ผู้ใช้งานฐานข้อมูลของ SQL Server ไม่โครซอฟท์แนะนำให้เลือกใช้กลุ่มของออบเจกต์ที่อาศัย SQL Server Data Provider โดยเฉพาะ เนื่องจากว่าผลทดสอบการทำงานที่ไมโครซอฟท์เผยแพร่ออกมา แสดงให้เห็นว่าถ้าใช้ OLEDB Data Provider ประเภท SQL Server จะทำงานช้ากว่า SQL Server Data Provider โดยตรงค่อนข้างมาก

2.3.5 กลุ่มออบเจกต์ที่ใช้ OLE Data Provider

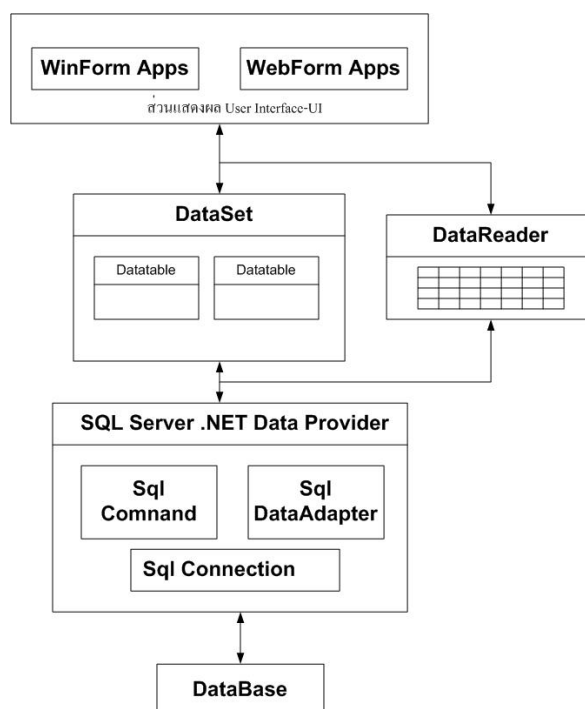
จะอาศัย OLE Data Provider ประเภทต่างๆ เป็นตัวกลางในการเข้าถึงข้อมูลในฐานข้อมูลแต่ละชนิดซึ่งรวมไปถึง SQL Server ด้วย เป็นแนวความคิดเดียวกับกลุ่มออบเจกต์ ADO ใน VB 6.0 ใช้คำนำหน้ากลุ่มออบเจกต์นี้ว่า OleDb เช่น OleDbConnection, OleDbDataAdapter เป็นต้น



รูปที่ 2-3 รูปแสดงแบบจำลองของกลุ่มออบเจกต์ ADO.NET ที่ใช้ OLEDB Provider

2.3.6 กลุ่มออบเจกต์ที่ใช้ SQL Server Data Provider

เป็นกลุ่มออบเจกต์ที่ถูกออกแบบมาสำหรับฐานข้อมูล SQL Server โดยเฉพาะ อาศัย SQL Server Provider เป็นตัวกลางในการเข้าถึงฐานข้อมูลของ SQL Server โดยเฉพาะ ใช้คำนำหน้ากลุ่มออบเจกต์นี้ว่า Sql เช่น SqlConnection, SqlDataAdapter เป็นต้นดังแสดงในรูป



รูปที่ 2-4 รูปแสดงแบบจำลองของกรุปออบเจกต์ ADO.NET ที่ใช้ SQL Server Provider โดยเฉพาะ

2.3.7 ข้อแตกต่างระหว่างออบเจกต์ DataReader กับออบเจกต์ DataSet

ออบเจกต์ OleDbDataReader และออบเจกต์ SqlDataReader ทำหน้าที่อ่านข้อมูลออกมาจากฐานข้อมูลโดยที่จะอ่านครั้งละ 1 เร็คคอร์ด มีสถานะเคลื่อนที่ไปข้างหน้า (Forward only) เพียงอย่างเดียว เกิดจากการคิวรีชุดคำสั่ง SQL ของออบเจกต์ OleDbCommand หรือ SqlCommand ส่วนออบเจกต์ DataSet เกิดจากการคิวรีชุดคำสั่ง SQL ของออบเจกต์ OleDbDataAdapter หรือ SqlDataAdapter

วิธีการใช้งานที่แตกต่างจากออบเจกต์ DataSet ก็คือ สมมุติว่าต้องการคิวรีข้อมูลหลายๆ เร็คคอร์ดที่มีปริมาณมากๆ แต่ไม่ต้องการอ่านทีเดียวทั้งหมด เข้ามาเก็บไว้ในหน่วยความจำ แต่ต้องการอ่านทีละ 1 เร็คคอร์ดก็พอ หรืออ่านด้วยปริมาณเร็คคอร์ดน้อยๆ นำมาใช้เป็นเงื่อนไขบางอย่าง ความต้องการเช่นนี้เหมาะสมเป็นอย่างยิ่งสำหรับการใช้งานออบเจกต์ OleDbDataReader และออบเจกต์ SqlDataReader ซึ่งจะส่งผลให้เป็นการประหยัดหน่วยความจำไปในตัวเลย

ส่วนออบเจกต์ DataSet จะอ่านข้อมูลที่เรากำหนดด้วยชุดคำสั่ง SQL ออกมา แล้วเก็บไว้ในหน่วยความจำ จากนั้นจะแสดงข้อมูลในแบบใดก็ได้ตามความต้องการ ซึ่งเหมาะสมกับข้อมูลที่มีปริมาณไม่มากนัก

2.3.8 หน้าที่ของออบเจกต์แต่ละชนิด สำหรับหน้าที่ของออบเจกต์แต่ละตัวมีดังนี้

ออบเจกต์ OleDbConnection และ SqlConnection ทำหน้าที่สร้างการเชื่อมต่อเข้ากับฐานข้อมูล โดยที่ OleDbConnection จะอาศัย OLEDB Provider ทำหน้าที่เป็นตัวกลางในการเข้าถึงฐานข้อมูล ส่วน SqlConnection จะอาศัย SQL Server Provider ทำหน้าที่เป็นตัวกลางในการเข้าถึงข้อมูลในฐานข้อมูล

2.3.9 ADO.NET Namespaces

เมื่อก้าวถึงเครื่อง ADO.NET แท้จริงแล้วเรากำลังพูดถึงเรื่องของ System.Data และ System.Data OleDb namespaces ในเนมสเปซทั้งสองนี้มีคลาสต่าง ๆ อยู่มากที่สามารถใช้เพื่อเข้าถึงข้อมูลจาก data source

ชนิดของ class ที่เกี่ยวข้องกับโครงงานระบบบริหารคลังข้อสอบ มีดังต่อไปนี้

- OleDbConnection
- OleDbDataAdapter
- OleDbCommand
- OleDbParameter
- DataSet
- DataView

เมื่อใดที่คุณต้องใช้คลาสที่มีชื่อนำหน้าด้วยคำว่า OleDb เราต้องนำเข้า System.Data.OleDb ด้วยโค้ดดังนี้

```
Imports System.Data.OleDb
```

ถ้าเราต้องการใช้คลาสสองตัวสุดท้ายในรายการข้างต้นนี้ เราต้องนำเข้าเนมสเปซ System.Data ด้วยโค้ดดังนี้

```
Imports System.Data
```

OleDbConnection

คลาส OleDbConnection เป็นหัวใจของคลาสที่จะได้อธิบายในหัวข้อนี้ ในฐานะที่มันเป็นตัวเชื่อมต่อกับ data source ของเรา ซึ่งในที่นี้จะเป็น SQL Server สำหรับ constructor ของคลาสนี้จะรับพารามิเตอร์แบบตัวเลือกอยู่หนึ่งตัวซึ่งก็คือ connection string

The Connection String

ตัว connection string เองมาจากพารามิเตอร์ต่าง ๆ Constructor จะกำหนดคุณสมบัติใหม่ของคลาสขึ้นด้วย connection string ที่กำหนดคุณยังสามารถเรียก constructor ได้โดยไม่ต้องมี connection string เลยด้วย ซึ่งจะให้ผล การกำหนดคุณสมบัติของคลาสใหม่ขึ้น และคุณยังจำเป็นต้องกำหนดพรีอเพอร์ตีต่าง ๆ ของคลาส OleDbConnection ก่อนที่คุณจะทำการเชื่อมต่อนั้น ๆ ได้ ดังนั้น จึงเป็นการดีกว่าที่กำหนดคลาสนี้ด้วยการใช้ connection string ซึ่งทำให้เราเขียนโค้ดน้อยลงโดยไม่ต้องกำหนดพรีอเพอร์ตีของคลาสหลังจากเรากำหนดมันขึ้น Connection string ที่คุณใช้ก็ขึ้นอยู่กับ data store ที่วางแผนจะเข้าถึงมัน อย่างไรก็ตาม ก็มีพรีอเพอร์ตีพื้นฐานบางตัวของ connection string ที่ใช้ได้กับ data store ทั่วไป ตารางถัดไปนี้แสดงถึงพรีอเพอร์ตีที่ใช้ได้ทั่วไปในการกำหนด connection string

```
Dim objConnection As OleDbConnection = New _
    OleDbConnection ("Provider = SQLOLEDB;" & _
        "Data Source = localhost; Initial Catalog = pubs;" & _
        "User ID = sa; Password = vbnetnet;")
```

ลองมาดูว่าการเข้าพารามิเตอร์ข้างต้นใน connection string เพื่อกำหนดคอบเจ็กต์การเชื่อมต่อ โค้ดบางส่วนถัดไปนี้ ใช้คลาส OleDbConnection เพื่อกำหนดคอบเจ็กต์การเชื่อมต่อที่ใช้ user ID และรหัสผ่านใน connection string

```
Dim objConnection As OleDbConnection = New _
    OleDbConnection ("Provider = SQLOLEDB;" & _
        "Data Source = localhost; Initial Catalog = pubs;" & _
        "User ID = sa; Password = vbnetnet ;")
```

Connection string ข้างต้นใช้ SQLOLEDB provider สำหรับการเข้าถึงฐานข้อมูล SQL Server พารามิเตอร์ Data Source กำหนดว่าฐานข้อมูลอยู่ในเครื่องใดคอด เนื่องจากเรากำหนดใช้ค่าเป็น localhost พารามิเตอร์ Initial Catalog กำหนดฐานข้อมูลที่เราต้องการเข้าถึง และในกรณีนี้ก็คือฐานข้อมูลชื่อ pubs ส่วนสุดท้าย พารามิเตอร์ User ID และ Password กำหนดถึง user ID และรหัสผ่านของผู้ใช้ที่กำหนดไว้ในฐานข้อมูล

การเปิด และปิดการเชื่อมต่อ

เมื่อเราได้กำหนดคอบเจ็กต์การเชื่อมต่อขึ้นแล้วด้วย connection string ตามที่แสดงในข้างต้นนี้ เราก็จะสามารถใช้เมธอดของคอบเจ็กต์ OleDbConnection เช่น Open และ Close ได้ ซึ่งจะเป็นตัวเปิด และปิดต่อการ เชื่อมต่อฐานข้อมูลที่กำหนดไว้ใน connection string นั้นจริง ๆ ดังแสดงในตัวอย่างโค้ดต่อไปนี้

‘ Open the database connection . . .

objConnection.Open()

. . .

‘ Close the database connection . . .

objConnection.Close()

การเชื่อมต่อผ่าน Integrated Security ใน Windows NT หรือ 2000

การส่งค่า user Id และรหัสผ่านต่อ SQL Server จะทำงานได้เป็นอย่างดี ถ้าคุณมี SQL Server authentication แต่จะเกิดอะไรขึ้นถ้า SQL Server security และ Windows security ถูกใช้ร่วมกัน และคุณได้เชื่อมต่อ SQL Server โดยใช้ Windows authentication? มาดูตัวอย่างที่ได้ใช้พารามิเตอร์ Integrated Security เพื่อเชื่อมต่อกับฐานข้อมูล ตัวอย่างนี้จะใช้ข้อมูล login ใน Windows NT/2000 ของเราเพื่อเชื่อมต่อกับฐานข้อมูล สิ่งนี้จะทำงานได้ก็ต่อเมื่อ NT account ของคุณได้รับอนุญาตให้เข้าถึงฐานข้อมูลได้

```
Dim objConnection As OleDbConnection = New _
    OleDbConnection("Provider = SQLOLEDB;" &_
        "Data Source = localhost; Initial Catalog = pubs;" &_
        "Integrated Security = SSPI;")
```

สังเกตว่าในตัวอย่างนี้เราได้กำหนดพารามิเตอร์ Integrated Security แทนที่จะใช้พารามิเตอร์ User ID และ Password สิ่งนี้เป็นตัวบอกต่อ provider ว่าให้เรียก security provider ในระบบปฏิบัติการเพื่อขยายต่อการเข้าถึงจากชื่อ login ของเรา แล้วใช้ชื่อ login นั้นในการยอมรับการเข้าถึงฐานข้อมูล

OleDbDataAdapter

คลาส OleDbDataAdapter ถูกใช้เพื่ออ่านและเขียนข้อมูลทั้งจากและเข้าไปใน OLE DB Data source และกำหนดด้วยประโยค SQL หรือ procedure name เพื่อใช้ทำงาน คลาส OleDbDataAdapter ไม่ได้จัดเก็บข้อมูลใดๆ เอาไว้จริงๆ แต่ถูกใช้เป็นสะพานข้อมูลระหว่างคลาส DataSet และ data store นั้น

พรีอเปอเรเตอร์ของ SelectCommand

เมื่อเราต้องการอ่านข้อมูลจาก data store เราต้องกำหนดพรีอเปอเรเตอร์ SelectCommand ของคลาส OleDbDataAdapter เสียก่อน พรีอเปอเรเตอร์นี้จะสร้างหรือ derived มาจากคลาส OleDbCommand และถูกใช้เพื่อการ

กำหนดว่าจะเลือกข้อมูลอะไร และเลือกอย่างไร เมื่อเรากำหนดพรีอปเพอร์ดี SelectCommand เราต้องกำหนดพรีอปเพอร์ดีอื่น ๆ รวมด้วย เพื่อให้โอปเจกต์ OleDbDataAdapter ทราบถึงวิธีการค้นคืนข้อมูลนี้ ตารางถัดไปแสดงพรีอปเพอร์ดีที่ใช้ทั่วไปใน SelectCommand

พรีอปเพอร์ดี	รายละเอียด
Connection	กำหนดคอนเนกต์ OleDbConnection ที่จะถูกใช้เพื่อเข้าถึง data store
CommandText	กำหนดประโยค SQL หรือ stored procedure name ที่จะใช้เพื่อเลือกข้อมูล
CommandType	กำหนดค่าที่ใช้เป็นตัวบอกวิธีการที่พรีอปเพอร์ดี CommandText จะถูกตีความหมาย

การใช้ SelectCommand เพื่อสั่งใช้งาน SQL String

โค้ดถัดไปนี้แสดงถึงค่ากำหนดทั่วไปสำหรับพรีอปเพอร์ดีเหล่านี้เมื่อสั่งงาน SQL string

```
' Declare an OleDbDataAdapter object ...
Dim objDataAdapter As New OleDbDataAdapter

' Set the SelectCommand to OleDbCommand class ...
objDataAdapter.SelectCommand = New OleDbCommand()

' Set the SelectCommand properties ...
objDataAdapter.SelectCommand.Connection = objConnection
objDataAdapter.SelectCommand.CommandText = _
    "SELECT au_lname, au_fname FROM authors" & _
    "ORDER BY au_lname, au_fname"
objDataAdapter.SelectCommand.CommandType = CommandType.Text
```

สิ่งแรกที่โค้ดนี้กระทำคือ การประกาศคอนเนกต์ที่สร้างมาจากคลาส OleDbAdapter จากนั้นเราได้กำหนด SelectCommand ด้วยคลาส OleDbCommand

สุดท้ายเราได้กำหนดค่าให้กับพรีอปเพอร์ดีต่าง ๆ เริ่มจากพรีอปเพอร์ดี Connection ด้วยค่าคอนเนกต์การเชื่อมต่อที่มีอยู่ ต่อมากำหนดพรีอปเพอร์ดี CommandText ด้วยประโยค SELECT แล้วเราก็กำหนดพรีอปเพอร์ดี CommandType เป็นค่าที่เป็นตัวกำหนดวิธีที่พรีอปเพอร์ดี CommandText จะถูกตีความหมาย ในกรณีนี้คือ Text เนื่องจากเราใช้ SQL String

การใช้ SelectCommand เพื่อส่งใช้งานด้วย Stored Procedure

โค้ดถัดไปนี้แสดงถึงวิธีที่เราสามารถใช้กำหนดพารามิเตอร์ให้ได้ว่า เมื่อต้องการจะส่งใช้งาน stored procedure ซึ่งมันเป็นกลุ่มของประโยค SQL ที่จัดเก็บอยู่ในฐานข้อมูลในชื่อที่ไม่ซ้ำกัน และจะถูกส่งใช้งานเป็นหน่วย ๆ stored procedure ในตัวอย่างนี้ถูกสร้างโดยใช้ประโยค SQL เดียวกับที่นิยามไว้ในโค้ดที่ผ่านมา

```
' Declare an OleDbDataAdapter object . . .

Dim objDataAdapter As New OleDbDataAdapter()

' Set the SelectCommand to the OleDbCommand class . . .

objDataAdapter.SelectCommand = New OleDbCommand()

' Set the SelectCommand properties . . .

objDataAdapter.SelectCommand.Connection = objConnection

objDataAdapter.SelectCommand.CommandText = 'usp_select_auther_titles'

objDataAdapter.SelectCommand.CommandType = CommandType.StoredProcedure
```

DataSet

คลาส DataSet ถูกใช้เพื่อจัดเก็บข้อมูลที่ค้นคืนมาจาก data store และจัดเก็บข้อมูลนี้ไว้ในหน่วยความจำบนไคลเอนต์ ออบเจกต์ DataSet ประกอบด้วยคอลเล็กชันของตาราง (Table) , ความสัมพันธ์ (relationships) และ constraints ที่ตรงกันกับข้อมูลที่อ่านมาจาก data store

ข้อมูลใน DataSet แท้จริงแล้วจะถูกตัดการเชื่อมต่อ (disconnected) จาก data store และคุณสามารถดำเนินการกับข้อมูลได้โดยเป็นอิสระจาก data store คุณสามารถจัดการข้อมูลในออบเจกต์ DataSet ได้ทั้งการเพิ่ม อัปเดต และลบเรคคอร์ดต่าง ๆ แล้วค่อยกำหนดการเปลี่ยนแปลงนี้ใส่กลับคืนเข้าไปใน data store ด้วยการใช้ออบเจกต์

OleDbDataAdapter

ข้อมูลในออบเจกต์ DataSet อยู่ในรูป XML ทำให้การส่งผ่านระหว่างออบเจกต์เป็นไปได้อย่างมีประสิทธิภาพอย่างยิ่ง ส่วน XML นี้คุณไม่ต้องลงมือแก้ไขอะไรกับมันโดยตรง การแก้ไขต่าง ๆ ทั้งหมดใน XML จะถูกกระทำผ่านจากในพร็อบเพอร์ตีและเมธอดต่าง ๆ ของคลาส DataSet

เนื่องจากข้อมูลถูกจัดเก็บในรูป XML และ DataSet ที่มีคอลเล็กชันของตาราง ความสัมพันธ์ และ constraints จึงมีรูปแบบของข้อมูลที่จัดเก็บเป็น XML ไปด้วย ซึ่งคุณก็ไม่ต้องมีความรู้เกี่ยวกับ XML เลยเพื่อที่จะใช้งาน DataSet เนื่องจาก DataSet มีข้อมูลจริง ๆ ที่เรียกค้นคืนจาก data store เราจึงจะเชื่อมต่อหรือ bind ส่วน DataSet นี้ไว้กับคอนโทรล เราจะเป็นเพิ่มเติมในส่วนต่อไปในบทนี้ คอนโทรลเหล่านี้จะให้ผลแสดงข้อมูลที่มีใน DataSet นั้น

DataView

คลาส DataView โดยทั่วไปจะถูกใช้เพื่อการจัดเรียงลำดับ (Sorting), การกรอง (Filtering), การค้นหา (Searching), การแก้ไข (Editing) และการสำรวจ (Navigating) ข้อมูลจาก DataSet ส่วน DataView นี้ ทำงานแบบ bindable ซึ่งหมายความว่า มันสามารถจะถูกผูกหรือ bound เข้ากับคอนโทรลต่าง ๆ แบบที่ DataSet ผูกเข้ากับคอนโทรลต่าง ๆ ได้นั่นเอง

ข้อมูลที่อยู่ในหน่วยความจำถูกใช้โดยออบเจกต์ DataSet และ DataView ก็คือ ออบเจกต์ DataTable คลาส DataView เป็นวิวที่ถูกปรับแต่งของ DataTable ซึ่งสร้างมาจากออบเจกต์ DataSet

คุณสามารถสร้าง DataView จากข้อมูลที่มีอยู่ใน DataTable ที่จะประกอบด้วยเฉพาะข้อมูลที่ต้องการแสดงเท่านั้น ข้อมูลใน DataView ถูกแยกจากข้อมูลที่มีอยู่ใน DataTable ของ DataSet หนึ่ง ๆ ดังนั้นคุณสามารถจัดการข้อมูลได้ โดยไม่มีผลกระทบต่อข้อมูลใน DataSet เลย

Constructor สำหรับคลาส DataView จะ initialize อินสแตนซ์ของคลาส DataView ขึ้นใหม่ และรับ DataTable เป็นอาร์กิวเมนต์ (Argument) โค้ดถัดไปนี้แสดงถึงการประกาศออบเจกต์ DataView และ initialize มันโดยใช้ตารางชื่อ authors จาก DataSet ชื่อ objDataSet สังเกตว่าเราได้เข้าถึงคอลเล็กชัน Tables ของออบเจกต์ DataSet โดยกำหนดที่พรีอเพอร์ติ์ Tables และชื่อของตาราง

```
' Set the DataView object to the DataSet object . . .
```

```
Dim objDataView = New DataView(objDataSet.Tables("authors"))
```

เมธอด Sort

เมื่อ DataView ถูก Initialized และมีข้อมูลเข้ามาแล้ว เราก็สามารถที่จะจัดการต่อข้อมูลนั้น ๆ ได้ ตัวอย่างเช่น สมมติว่าต้องการจัดเรียงลำดับใหม่ต่างจากที่มีใน DataSet ซึ่งต้องกำหนดจากพรีอเพอร์ติ์ Sort และกำหนดคอลัมน์ ซึ่งอาจมากกว่าหนึ่งที่จะใช้จัดเรียงลำดับ โค้ดถัดไปนี้แสดงถึงการจัดเรียงลำดับใน DataView โดยใช้ชื่อ และนามสกุลของผู้เขียนเอง

```
objDataView.Sort = "au_fname, au_lname"
```

โดยค่าปกติการจัดเรียงลำดับไม่ว่าจะเป็นในส่วนใด ๆ ก็จะเป็นแบบเรียงจากน้อยไปมาก หรือ ascending และใน DataView ก็เป็นเช่นนั้นด้วย ถ้าต้องการจัดเรียงแบบมากไปน้อยหรือ descending ก็ต้องกำหนดคีย์เวิร์ด (key word) DESC ดังแสดงในโค้ดต่อไปนี้

```
objDataView.Sort = "au_fname, au_lname DESC"
```

เมธอด Find

ถ้าต้องการค้นหาแถวของข้อมูลใน DataView เราจะใช้เมธอด Find ซึ่งเมธอดนี้จะค้นหาข้อมูลในคอลัมน์ที่เป็น primary key ของ DataView ดังนั้นก่อนที่จะใช้เมธอด Find เราต้องจัดเรียงลำดับคอลัมน์ใน DataView ที่มีข้อมูลที่เราต้องการจะค้นหาเสียก่อน คอลัมน์ใน DataView ที่ถูกจัดเรียงลำดับจะกลายมาเป็นคอลัมน์ primary key ในออปเจกต์ DataView

ตัวอย่างเช่น สมมติว่าเราต้องการจะหาผู้เขียนที่มีชื่อว่า Ann เราก็จำเป็นต้องจัดเรียงลำดับไป DataView ด้วยชื่อเพื่อกำหนดคอลัมน์นี้เป็นคอลัมน์ primary key ใน DataView แล้วจึงใช้เมธอด Find ดังแสดงในโค้ดถัดไปนี้

```
objDataView.Sort = "au_fname"
intPosition = objDataView.Find("Ann")
```

ถ้าค้นหาพบเมธอด Find จะให้ผลลัพธ์เป็นตำแหน่งของเรคอร์ดใน DataView มิฉะนั้นแล้ว DataView ก็จะให้ผลลัพธ์เป็น -1 เพื่อเป็นตัวบอกว่าหาไม่พบ ถ้าเมธอด Find หาพบแล้ว มันจะหยุดการค้นหาและให้ผลลัพธ์เฉพาะตำแหน่งแรกที่พบ

เมธอด Find ทำงานโดยไม่สนใจตัวพิมพ์ใหญ่ – เล็ก (not – case-sensitive) ซึ่งหมายถึงหากจะค้นหานักเขียนชื่อ Ann คุณก็สามารถป้อนด้วยข้อความ Ann หรือ ann ก็ได้

เมธอด Find จะค้นหาตามข้อความทั้งหมดที่ป้อนเข้ามา ตัวอย่างเช่น หากจะค้นหานักเขียนที่มีนามสกุลเป็น Del Castillo จะป้อนเพียง Del เพื่อให้ค้นหาไม่ได้ ต้องป้อนครบทั้งข้อความ ซึ่งจะกระทำได้ด้วยโค้ดถัดไปนี้ ดังแสดงในตัวอย่างนี้ เราได้กำหนดค่าด้วยตัวพิมพ์เล็ก ซึ่งก็จะทำงานได้เป็นอย่างดี และหาผลลัพธ์ ได้ตรงกับที่ต้องการ

```
objDataView.Sort = "au_lname"
intPosition = objDataView.Find("del castillo")
```

ฟร้อปเพอร์ตี้ RowFilter

เมื่อเรา initialize ส่วน DataView เราสามารถกรองแถวข้อมูลต่าง ๆ ที่มีอยู่ด้วยฟร้อปเพอร์ตี้ RowFilter เป็นตัวกำหนดเงื่อนไขที่จะใช้กับ DataView โดยใช้ชื่อคอลัมน์ตามด้วยตัวดำเนินการ และค่า ค่านี้ต้องใส่อยู่ในเครื่องหมายคำพูดแบบซิงเกิล (single quote) โค้ดต่อไปนี้จะค้นหาเฉพาะผู้เขียนที่มีนามสกุลเป็น Green

```
' Set the DataView object to the DataSet object . . .
objDataView = New DataView(objDataSet.Tables("authors"))
objDataView = RowFilter = "au_lname = 'Green'"
```

หากต้องการค้นคืนเอาแถวข้อมูลนักเขียนทุกคนยกเว้นที่มีนามสกุลเป็น Green ก็ต้องใช้ตัวดำเนินการ “not equal to” ดังนี้

```
‘ Set the DataView object to the DataSet object . . .
```

```
objDataView = New DataView (objDataSet.Table (“authors”))
```

```
objDataView = RowFilter = “au_lname <> ‘Green’”
```

OleDbCommand

คลาส OleDbCommand แทนถึงคิวรีที่สร้างไว้ต่อ data store คิวรีสามารถใช้เลือก แทรกวาง อัปเดต หรือลบ คิวรี และสามารถแทนด้วย SQL string หรือ stored procedure คิวรีที่จะนำมาส่งใช้งานอาจจะมีพารามิเตอร์ต่าง ๆ หรือไม่มีก็ได้ เราได้กล่าวถึงคลาส OleDbCommand ไปแล้วในหัวข้อ OleDbDataAdapter ว่าพร้อมเพอร์ดี SelectCommand ถูกสร้างมาจากคลาสนี้ ในหัวข้อนี้จะมาดูที่การใช้คลาสนี้เพื่อส่งใช้งานคิวรีต่อฐานข้อมูลนี้โดยไม่ต้องใช้ OleDbDataAdapter

Constructor สำหรับคลาส OleDbCommand มีอยู่หลากหลายรูปแบบ แต่เมธอดที่ง่ายที่สุดของมันก็คือ การ initialized ออบเจกต์ OleDbCommand โดยไม่มีพารามิเตอร์ แล้วเมื่อออบเจกต์ initialized แล้ว เราก็สามารถ กำหนดพร้อมเพอร์ดีต่าง ๆ ที่ต้องใช้งานได้ โค้ดถัดไปนี้แสดงถึงการ initialize ออบเจกต์ OleDbCommand

```
Dim objCommand As OleDbCommand = New OleDbCommand()
```

พร้อมเพอร์ดี Connection

เช่นเดียวกับออบเจกต์ OleDbDataAdapter มีพร้อมเพอร์ดีอยู่ส่วนหนึ่งที่จำเป็นต้องถูกกำหนดค่าก่อนที่จะเราจะส่งใช้งานคิวรีได้ พร้อมเพอร์ดีแรกก็คือ Connection พร้อมเพอร์ดีตัวนี้กำหนดค่าด้วยออบเจกต์ที่เป็นการเชื่อมต่อที่แยกที่พ้อยกับ data store เช่น ออบเจกต์ OleDbConnection ดังโค้ดต่อไปนี้

```
objCommand.Connection = objConection
```

พร้อมเพอร์ดีถัดไปที่ต้องถูกกำหนดค่าให้ก็คือพร้อมเพอร์ดี CommandText พร้อมเพอร์ดีนี้กำหนดด้วย SQL string หรือ stored procedure ต่อจากพร้อมเพอร์ดีนี้ก็คือพร้อมเพอร์ดี CommandType เพื่อเป็นตัวกำหนดวิธีที่พร้อมเพอร์ดี CommandText จะถูกตีความหมาย โค้ดถัดไปนี้แสดงถึงการกำหนดค่าพร้อมเพอร์ดีทั้งสองนี้

SQL Server บังคับให้เราใส่คำสั่งลงในเครื่องหมายคำพูดแบบชิดเดียวอย่างที่กระทำในที่นี้ เรากำลังกับค่าที่แทรกจากเท็กซ์บ็อกซ์บนฟอร์ม และกำหนดมันด้วยประโยค INSERT ของเรา ถ้าเราต้องแทรกวงค่าจำนวนเต็ม ก็ไม่ต้องใช้เครื่องหมายคำพูดแบบชิดเดียวนี้อีก

```
objCommand.Connection = objConnection
objCommand.CommandText = "INSERT INTO authors" &_
    "(au_id, au_lname, au_fname)" &_
    "VALUES(' " & txtAuthorID.Text & " ', " &_
objCommand.CommandType = CommandType.Text
```

ประโยค INSERT ที่แสดงในข้างต้นนี้เป็นประโยคง่าย ๆ ไม่ซับซ้อน และทำงานได้เป็นอย่างดีกับค่าแบบ Integer และ String แต่จะเกิดอะไรขึ้น เมื่อเราต้องการแทรกวงค่าที่เป็นชนิดอื่น ๆ เช่น Currency? เราต้องสร้างช่องกรอกค่า (placeholder) ในประโยค SQL และส่งข้อมูลนั้นเป็นพารามิเตอร์ให้

คอลเล็กชัน Parameters

ช่องกรอกค่า หรือ Placeholders ในประโยค SQL ก็คือ เครื่องหมายคำถามที่จะใช้เติมค่าด้วยพารามิเตอร์ต่าง ๆ สิ่งนี้จำเป็นต้องใช้กับข้อมูลบางชนิด เช่น Decimal และ Currency เราไม่สามารถกำหนดค่าให้กับข้อมูลชนิดนี้โดยใช้ตรรกೆಯารูปแบบข้างต้นได้ รูปแบบนั้นจะทำงานได้เฉพาะกับสตริง จำนวนเต็ม วันเวลา และค่าบูลีน (Boolean)

```
objCommand.CommandText = "INSERT INTO titles" &_
    "(title_id, title, type, price, pubdate)" &_
objCommand.CommandType = CommandType.Text
```

สิ่งที่เราจำเป็นต้องกระทำถัดไปก็คือการสร้างพารามิเตอร์ที่จะถูกใช้เพื่อแทรกวงค่าต่าง ๆ เข้าใน placeholders เมื่อประโยค SQL ถูกส่งใช้งาน เราจำเป็นต้องสร้าง และเพิ่มพารามิเตอร์ต่าง ๆ เข้าในคอลเล็กชัน Parameters ของออบเจกต์ OleDbCommand เมื่อเราพูดถึงพารามิเตอร์ ในที่นี้เรากำลังหมายถึง พารามิเตอร์ต่าง ๆ ที่ต้องการเป็นข้อมูลในประโยค SQL หรือ stored procedure ของเรา ไม่ใช่พารามิเตอร์ที่ต้องส่งให้กับเมธอดหนึ่ง

เราสามารถเข้าถึงคอลเล็กชัน parameters ของออบเจกต์ OleDbCommand โดยการกำหนดพรีอ็อปเพอร์ดี Parameters เมื่อเราเข้าถึงคอลเล็กชัน Parameters แล้ว เราสามารถใช้พรีอ็อปเพอร์ดีและเมธอดของมันเพื่อสร้างพารามิเตอร์ต่าง ๆ ขึ้นในคอลเล็กชันได้

เราใช้ Parameters constructor เพื่อสร้าง และเพิ่ม Parameter ใหม่เข้าในคอลเล็กชัน Parameters ดังแสดงด้วยโค้ดถัดไปนี้ Parameters constructors มีรูปแบบอยู่หลากหลาย แต่เมธอดที่ง่ายที่สุดก็คือที่ใช้เพื่อ initialize ออบเจกต์ OleDbParameter โดยไม่มีพารามิเตอร์ดังนี้

```
objCommand.ParametersAdd(New OleDbParameter())
```

พรีอปรเพอร์ดี Direction

เมื่อเราได้เพิ่ม Parameter หนึ่ง ๆ เข้าในคอลเล็กชัน Parameters แล้ว ก็จะมีพรีอปรเพอร์ดีอยู่จำนวนหนึ่งที่เรากำลังกำหนดค่าต่อ Parameter พรีอปรเพอร์ดีตัวแรกก็คือพรีอปรเพอร์ดี Direction พรีอปรเพอร์ดีนี้มีค่ามาตรฐานของ Input เพื่อเป็นตัวกำหนด direction ของพารามิเตอร์ที่จะใช้ จริง ๆ แล้วเราไม่ได้จำเป็นต้องกำหนดพารามิเตอร์นี้ อย่างไรก็ตาม เราพบว่ากำหนัดพารามิเตอร์นี้ช่วยให้เราอธิบายการเขียนโค้ดได้สะดวกขึ้น

```
objCommand.ParametersItem(0).Direction = ParameterDirection.Input
```

สังเกตว่าในโค้ดข้างต้นเรากำหนดตำแหน่งของพารามิเตอร์ในคอลเล็กชันที่ (0) หากต้องการ จะกำหนดเป็นชื่อของพารามิเตอร์นี้แทนก็ได้ดังแสดงถัดไป สำหรับ input parameters แล้ว จริง ๆ ก็ไม่จำเป็นต้องทำเช่นนี้เลย เพราะว่าเมื่อเรากำหนดค่าพรีอปรเพอร์ดีให้กับมันแล้ว ก็จะไม่ต้องเข้าถึงมันอีกเลย

พรีอปรเพอร์ดี ParameterName

สำหรับส่วน output parameters ก็เป็นเรื่องสมเหตุสมผลที่จำกำหนดชื่อให้กับพารามิเตอร์ เหตุผลก็คือเราต้องกำหนดพารามิเตอร์ที่จะเรียกใช้ประโยค SQL แล้วประโยคนั้น ๆ ก็จะถูกส่งใช้งาน และเราก็จะเข้าถึง output parameter เพื่อเรียกค้นคืนค่าที่เราวางมันไว้ในประโยค SQL

เพื่อที่จะกำหนดชื่อให้กับพารามิเตอร์ เราสามารถกำหนดพรีอปรเพอร์ดี ParameterName ได้ดังนี้

```
objCommand.Parameters.Item(0).ParameterName = "AuthorID"
```

แล้วคุณก็สามารถเข้าถึง พารามิเตอร์นี้ได้จากชื่อของมัน แทนที่จะต้องมาจำ หรือทราบลำดับการอ้างอิงของมันดังนี้

```
objCommand.Parameters.Item("AuthorID").Direction = _  
ParameterDirection.Input
```

พรีอเปอเรเตอร์ DBType

พรีอเปอเรเตอร์ที่ถัดไปของ Parameter ที่เราต้องกำหนดก็คือพรีอเปอเรเตอร์ DBType พรีอเปอเรเตอร์นี้จำกำหนดชนิดพื้นฐานของพารามิเตอร์นี้ ซึ่งเป็นชนิดข้อมูลตามที่ใช้ใน data store และในกรณีของเรานี้ก็คือ SQL Server

โค้ดถัดไปแสดงถึงพรีอเปอเรเตอร์ DBType ที่ได้ถูกกำหนดด้วยพารามิเตอร์สองตัวที่ต่างกัน สังเกตว่าโค้ดบรรทัดแรกเป็นการกำหนดชนิดข้อมูลเป็น String ซึ่งใช้สำหรับสตริง และบรรทัดที่สองเป็นการกำหนดชนิดข้อมูลเป็น Currency ซึ่งจะถูกใช้เกี่ยวกับค่าการเงิน

```
objCommand.Parameter.Item(0).DBType = DbType.String
objCommand.Parameter.Item(1).DBType = DbType.Currency
```

พรีอเปอเรเตอร์ Size

พรีอเปอเรเตอร์ที่ถัดไปที่ต้องกำหนดก็คือ พรีอเปอเรเตอร์ Size พรีอเปอเรเตอร์นี้เป็นตัวกำหนดขนาดสูงของข้อมูลที่อยู่ในพารามิเตอร์นี้ ซึ่งทำงานได้เป็นอย่างดีกับค่าแบบสตริง สำหรับข้อมูลชนิดอื่น ๆ เช่น Integer และ Currency มันจะเป็นไปตามขนาดชนิดข้อมูลตามที่กำหนดไว้ใน data store

โค้ดสองบรรทัดต่อไปนี้จะกำหนดค่าข้อมูลแบบ String ซึ่งมีค่าเป็นสตริง และพรีอเปอเรเตอร์ Size โดยกำหนดให้มีค่าสูงสุดที่หกตัวอักษร

โค้ดอีกสองบรรทัดสุดท้ายกำหนด ชนิดข้อมูล Currency และขนาดของพารามิเตอร์ Size เป็น 8 ซึ่งเป็นขนาดใน SQL Sever สำหรับชนิดข้อมูลแบบ Currency

```
objCommand.Parameters.Item(0).DBType = DbType.String
objCommand.Parameters.Item(1).Size = 6
objCommand.Parameters.Item(1).DBType = DbType.Currency
objCommand.Parameters.Item(1).Size = 8
```

พรีอเปอเรเตอร์ Value

พรีอเปอเรเตอร์ตัวสุดท้ายที่ต้องกำหนดก็คือพรีอเปอเรเตอร์ Value ส่วนนี้เป็นการกำหนดค่าจริง ๆ เข้าใน Parameter นี้ในโค้ดถัดไปนี้ เรากำลังกำหนดว่า ค่าถูกค้นคืนจากเท็กซ์บ็อกซ์ชื่อ txtprice และใช้ค่าในพรีอเปอเรเตอร์ Text ของเท็กซ์บ็อกซ์

```
objCommand.Parameters.Item(2).Value = txtPrice.Text
```

เมธอด ExecuteNonQuery

เมื่อเราได้กำหนดพารามิเตอร์ทั้งหมดในคอลเล็กชัน Parameters แล้ว เราสามารถส่งใช้งานเมธอด ExecuteNonQuery ของออปเจกต์ OleDbCommand เมธอดนี้จะส่งใช้งานประโยค SQL และเป็นตัวกำหนดให้แทรกวางเข้าในฐานข้อมูล โค้ดตัวอย่างการใช้เมธอดนี้ เช่น

```
objCommand.ExecuteNonQuery()
```